



Tell me in detail that to build this or these types of projects what should I learn in depth? from zero. explain in detail.

You do not need to become an electrical engineer, AI researcher, cybersecurity expert, and full-stack developer all at once. For this FYP, you need a **practical understanding of several connected areas** and deep implementation skills in a smaller number of them.

Since you already have experience with [ASP.NET](#) Core, Java, databases, and web development, your main learning gap will probably be:

1. Python for data and AI.
2. Machine learning and time-series analysis.
3. Federated Learning.
4. Basic smart-grid concepts.
5. Explainable AI and security.
6. Integrating AI with your web dashboard.

The project should be built gradually. Do not begin with agents, cybersecurity, dashboards, and federated learning simultaneously.

1. First understand the project concept

Before writing code, understand this simplified scenario:

Several virtual substations collect electricity data.

Substation A has residential data.

Substation B has high-demand data.

Substation C has solar and household data.

Substation D has industrial-like data.

Each substation trains an AI model using its own local data.

The substations send model updates—not raw electricity data—to a federated server.

The server combines the updates and creates a better global model.

The global model predicts demand, detects abnormalities, and estimates equipment failure.

AI agents convert those predictions into understandable recommendations.

A web dashboard displays everything to the operator.

The entire system is therefore a combination of:

- Data Engineering
- + Machine Learning
- + Federated Learning
- + Simulation
- + Explainable AI
- + Cybersecurity
- + Web Development

Federated Learning is commonly described as moving the computation to the data instead of moving the data to a central training location. The Flower framework provides tutorials and infrastructure for building federated systems with frameworks such as PyTorch, TensorFlow, scikit-learn, and others. ^[1] ^[2]

2. Learning roadmap

A realistic learning sequence is:

- Phase 1: Programming and tools
- Phase 2: Mathematics and statistics
- Phase 3: Data handling
- Phase 4: Machine learning
- Phase 5: Time-series forecasting
- Phase 6: Anomaly detection
- Phase 7: Predictive maintenance
- Phase 8: Smart-grid fundamentals
- Phase 9: Federated Learning
- Phase 10: Explainable AI
- Phase 11: Security
- Phase 12: Agents and backend integration
- Phase 13: Dashboard and deployment

You can learn some phases in parallel, but the order above reduces confusion.

3. Phase 1: Programming foundation

3.1 Learn Python

Although your web application can remain in ASP.NET Core, Python is the most practical language for data science and machine learning.

You should learn:

Basic Python

- Variables and data types.
- Strings, lists, tuples, sets, and dictionaries.
- Conditions and loops.
- Functions.
- Classes and objects.
- Exception handling.
- Reading and writing files.
- Modules and packages.
- Virtual environments.
- JSON and CSV processing.

Example:

```
substation = {  
    "id": "A",  
    "load_kw": 82.5,  
    "voltage": 230.1,  
    "status": "normal"  
}  
  
if substation["load_kw"] > 100:  
    substation["status"] = "warning"
```

Python libraries

You should become comfortable with:

- NumPy for numerical operations.
- pandas for tables and time-series data.
- matplotlib and seaborn for visualization.
- scikit-learn for traditional machine-learning models.
- PyTorch or TensorFlow for neural networks.
- joblib for saving traditional models.
- FastAPI or Flask for exposing Python models through APIs.

You do not need to abandon C# or Java. A good architecture for you could be:

```
ASP.NET Core  
↓  
Web dashboard and main application API  
  
Python FastAPI  
↓
```

AI and federated-learning services

PostgreSQL

↓

Results, users, alerts, and metadata

3.2 Learn Git and Linux

You should know how to:

```
python -m venv .venv
pip install pandas scikit-learn
git init
git add .
git commit -m "Initial project"
```

Also learn:

- Environment variables.
- Process management.
- Ports.
- Logs.
- Basic Docker usage.
- Running services on Linux.
- `curl` for API testing.
- File permissions.
- SSH.

These skills will help when you run:

```
Federated server
Substation client A
Substation client B
Substation client C
Substation client D
AI API
ASP.NET application
```

4. Phase 2: Mathematics and statistics

You do not need advanced university-level mathematics for the first prototype, but you must understand the ideas behind the models.

4.1 Descriptive statistics

Learn:

- Mean.
- Median.
- Minimum and maximum.
- Variance.
- Standard deviation.
- Percentiles.
- Correlation.
- Distribution.
- Outliers.

For example, if normal voltage is approximately 230 V but one reading is 160 V, that reading may be an anomaly.

4.2 Probability

Understand:

- Probability.
- Conditional probability.
- Probability distributions.
- Classification probability.
- Confidence score.
- False positive and false negative.

If your model reports:

```
Failure probability = 0.87
```

this usually means the model estimates an 87% probability according to its learned behavior. It does not automatically mean that the transformer will certainly fail.

4.3 Linear algebra

Learn only the practical concepts initially:

- Vectors.
- Matrices.
- Dimensions.
- Matrix multiplication.
- Model parameters.

- Feature vectors.
- Weight updates.

A machine-learning model receives a feature vector such as:

```
[  
    temperature,  
    vibration,  
    load_percentage,  
    voltage,  
    operating_hours  
]
```

The model uses weights to transform these values into a prediction.

4.4 Calculus and optimization

Understand:

- Loss function.
- Gradient.
- Gradient descent.
- Learning rate.
- Epoch.
- Batch.
- Overfitting.
- Regularization.

Federated Learning also depends on local model optimization. Each client adjusts model weights locally, and the server aggregates those updated weights.

5. Phase 3: Data engineering

This is one of the most important parts of the project. A strong model cannot fix badly prepared data.

5.1 Understand dataset structure

The main electricity dataset contains time-based measurements such as power, voltage, current, reactive power, and sub-metering values. For renewable and advanced residential scenarios, Pecan Street data may include household consumption, solar generation, EV charging, and circuit-level measurements. NREL also provides grid-related datasets and synthetic distribution-system models that can be useful for more realistic grid experiments. ^[3]

^[4] ^[5]

A typical record may look like:

timestamp	active_power	voltage	current
2007-01-01 00:00:00	1.300	234.8	5.4
2007-01-01 00:01:00	1.280	235.1	5.3

5.2 Learn data cleaning

You should learn how to handle:

- Missing values.
- Duplicate rows.
- Incorrect timestamps.
- Invalid numerical values.
- Different units.
- Outliers.
- Unequal sampling intervals.
- Time zones.
- Daylight-saving changes, when relevant.
- Sensor dropouts.

Example:

```
df["timestamp"] = pd.to_datetime(df["timestamp"])
df = df.sort_values("timestamp")
df = df.drop_duplicates(subset=["timestamp"])
```

5.3 Learn resampling

The raw data may be recorded every minute, while your model may use five-minute or fifteen-minute intervals.

```
df = df.set_index("timestamp")
five_minute_data = df.resample("5min").mean()
```

You should understand that resampling can use:

- Mean.
- Sum.
- Minimum.
- Maximum.
- Forward fill.

For power measurements, average power and total energy must not be treated as the same thing.

5.4 Learn feature engineering

Create useful inputs from timestamps:

```
hour
minute
day_of_week
day_of_month
month
is_weekend
is_holiday
```

Create lag features:

```
load_previous_5_minutes
load_previous_15_minutes
load_previous_1_hour
load_previous_24_hours
```

Create rolling features:

```
rolling_mean
rolling_std
rolling_min
rolling_max
```

For a forecasting model:

```
Input:
previous load values
previous voltage values
hour
day of week
rolling average

Output:
future load
```

5.5 Understand data leakage

Data leakage occurs when information from the future accidentally enters the training data.

For time-series projects, do not randomly mix future and past records.

Use chronological splitting:

```
First 70% → training
Next 15% → validation
Final 15% → testing
```

This is more realistic than randomly selecting rows.

6. Phase 4: Machine learning fundamentals

Before Federated Learning, first learn normal centralized machine learning.

6.1 Supervised learning

You have:

```
Input features → known target → model learns relationship
```

Example:

```
temperature, vibration, load → fault label
```

Common algorithms:

- Linear regression.
- Decision tree.
- Random forest.
- Gradient boosting.
- Logistic regression.
- Support vector machine.
- Neural networks.

For an FYP, begin with baseline models:

Forecasting:

- Linear Regression
- Random Forest
- XGBoost or LightGBM
- LSTM later

Predictive maintenance:

- Logistic Regression
- Random Forest
- XGBoost

Anomaly detection:

- Isolation Forest
- Autoencoder

6.2 Training concepts

Learn:

- Features X .
- Target y .
- Training.
- Validation.
- Testing.
- Prediction.
- Loss.
- Metrics.
- Hyperparameters.
- Cross-validation.

For time-series data, normal random cross-validation is not always appropriate. Use time-aware validation.

6.3 Model evaluation

For demand forecasting, learn:

- MAE.
- MSE.
- RMSE.
- MAPE.

For classification, learn:

- Accuracy.
- Precision.
- Recall.
- F1-score.
- Confusion matrix.
- ROC-AUC.

For anomaly detection, learn:

- Precision of alerts.
- Recall of known injected anomalies.
- False-alert rate.
- Detection delay.

Example:

```
Actual demand:      100 kW
Predicted demand:    94 kW
Absolute error:      6 kW
```

Never show only accuracy. An advisor may ask whether your model actually improves forecasting or fault detection.

7. Phase 5: Time-series forecasting

This is the first AI task you should implement.

7.1 What is a time series?

A time series is data recorded in time order:

```
09:00 → 80 kW
09:01 → 82 kW
09:02 → 81 kW
09:03 → 85 kW
```

The order matters. The model should use earlier observations to predict later observations.

7.2 Forecasting target

Choose a specific target, such as:

```
Predict the next 15-minute load.
```

Avoid saying only "we predict electricity." Define:

- What you predict.
- How far into the future.
- At what interval.
- For which substation.
- Using which input features.

A manageable first target is:

```
Input: previous 60 minutes
Output: next 15-minute active power
```

7.3 Models to learn

Baseline model

Use the previous value:

```
next load = current load
```

This is called a persistence baseline. Your AI model should beat it.

Traditional models

Learn the idea of:

- Moving average.
- ARIMA.
- SARIMA.
- Random Forest regression.
- Gradient boosting.

Neural networks

Later learn:

- Dense neural networks.
- LSTM.
- GRU.
- 1D CNN.
- Transformer-based forecasting, only if time permits.

Do not begin with Transformers. An LSTM or gradient-boosting model is much easier to explain and deploy.

7.4 Forecasting flow

```
Raw readings
  ↓
Clean timestamps
  ↓
Resample data
  ↓
Create lag features
  ↓
Split chronologically
  ↓
Train local model
  ↓
```

```
Predict future load
↓
Calculate MAE/RMSE
↓
Display actual vs predicted chart
```

7.5 What the dashboard displays

```
Substation A
Current load: 82 kW
Predicted load in 15 minutes: 91 kW
Forecast status: Increasing
Forecast error: 4.8 kW
```

Chart:

```
Actual load      _____
Predicted load   - - - - -
```

8. Phase 6: Anomaly detection

An anomaly is a reading or behavior that is significantly different from normal behavior.

Examples:

```
Voltage suddenly drops.
Current becomes unusually high.
Power consumption suddenly spikes.
A sensor stops changing.
Several readings become physically inconsistent.
```

8.1 Two approaches

Statistical detection

Calculate a normal range:

```
mean ± 3 × standard deviation
```

A value outside the range is flagged.

Machine-learning detection

Train a model on normal data:

- Isolation Forest.
- One-Class SVM.
- Autoencoder.
- Local Outlier Factor.

For your first version, Isolation Forest is easier to implement and explain.

8.2 Feature examples

```
active_power
reactive_power
voltage
current
power_factor
rolling_mean
rolling_standard_deviation
```

You can also create physical consistency checks. For example:

```
power  $\approx$  voltage  $\times$  current  $\times$  power factor
```

The exact relationship depends on whether the system is single-phase or three-phase, so describe this as a simplified simulation unless you model the electrical system accurately.

8.3 Anomaly detection flow

```
Normal historical data
  ↓
Train anomaly detector
  ↓
New sensor readings
  ↓
Calculate anomaly score
  ↓
Compare with threshold
  ↓
Normal or suspicious
  ↓
Create alert
```

Example:

```
Substation C
Voltage: 185 V
Anomaly score: 0.94
```

```
Status: Critical anomaly
Reason: voltage significantly below learned normal range
```

8.4 Avoid a common mistake

Do not call every anomaly a cyberattack.

An unusual value could be:

- A sensor fault.
- Equipment failure.
- A sudden legitimate load.
- A communication error.
- A cyberattack.

Use careful terminology:

```
Detected anomaly
```

Then the Security Agent can investigate whether it resembles a cyber event.

9. Phase 7: Predictive maintenance

Predictive maintenance means estimating equipment health before a failure occurs.

9.1 What equipment are you simulating?

Choose one equipment type, preferably a transformer.

Your synthetic equipment table might contain:

```
timestamp
transformer_id
substation_id
temperature
vibration
load_percentage
voltage
current
oil_temperature
operating_hours
equipment_age
maintenance_count
health_status
fault_label
```

9.2 Create realistic states

Use clear states:

```
0 = Normal
1 = Warning
2 = Critical
3 = Failure risk
```

Example:

Temperature	Vibration	Load	Status
55°C	1.2	60%	Normal
65°C	2.0	75%	Warning
78°C	3.4	87%	Critical
92°C	5.8	97%	Failure risk

These values are simulation assumptions. You must document how the values were generated and avoid presenting them as real measurements.

9.3 Fault-injection logic

A simple degradation function could be:

```
temperature increases gradually
vibration increases gradually
load stress increases
health score decreases
fault probability increases
```

For example:

```
health_score = (
    100
    - 0.5 * temperature_excess
    - 5.0 * vibration
    - 0.2 * load_percentage
)
```

This is not a real transformer physics model. It is a controlled simulation rule for evaluating your software architecture.

9.4 Models

Start with:

- Logistic regression.
- Random forest classifier.
- Gradient boosting.
- Regression model for health score.

Then compare:

```
Actual label vs predicted label
```

9.5 Predictive-maintenance output

```
Transformer: T-17
Health state: Warning
Failure risk: 87%
Top factors:
- High temperature
- Increasing vibration
- High loading
Recommendation:
Schedule inspection
```

10. Phase 8: Smart-grid fundamentals

You do not need to design a complete electrical grid. You do need enough domain knowledge to use the terminology correctly.

10.1 Learn the electricity path

Understand this simplified structure:

```
Generation
  ↓
Transmission
  ↓
Substation
  ↓
Distribution network
  ↓
Consumers
```

10.2 Learn important electrical terms

Study:

- Voltage.
- Current.
- Active power.
- Reactive power.
- Apparent power.
- Energy.
- Frequency.
- Power factor.
- Load.
- Peak demand.
- Renewable generation.
- Net load.
- Transformer.
- Feeder.
- Bus.
- Circuit breaker.
- SCADA.
- PMU.
- IoT sensor.

10.3 Power versus energy

This distinction is important:

- **Power** is the rate of electricity use, commonly measured in watts or kilowatts.
- **Energy** is electricity consumed over time, commonly measured in watt-hours or kilowatt-hours.

Your dataset may contain power readings, while your dashboard may calculate energy consumption over an interval.

10.4 Understand substations

A substation can:

- Change voltage levels.
- Connect transmission and distribution networks.
- Monitor current and voltage.

- Protect equipment.
- Route electricity through feeders.
- Contain transformers and switching equipment.

For your FYP, each virtual substation is a software client with:

```
local data
local model
local equipment
local alerts
```

10.5 Learn renewable integration

For solar, understand:

```
Net load = consumer demand - solar generation
```

Example:

```
Demand = 82 kW
Solar generation = 35 kW
Net load = 47 kW
```

When solar generation falls, net demand may rise quickly. This creates an interesting forecasting problem.

11. Phase 9: Federated Learning

This is the core research component of the project.

11.1 Centralized learning

In centralized learning:

```
All data → central server → model training
```

Advantages:

- Easier to implement.
- One training dataset.
- Simple evaluation.

Disadvantages:

- Raw data must be collected centrally.
- Privacy concerns.

- Data ownership concerns.
- Communication of large datasets.

11.2 Federated Learning

In Federated Learning:

```
Server sends model → clients
Clients train locally
Clients send model updates → server
Server aggregates updates
Server sends improved model back
```

Raw local data stays at the client.

Flower's official learning material describes this client-server process and provides support for implementing federated systems and custom strategies. ^[2] ^[1]

11.3 Core terms

Client

A participating data owner or computing node.

In your project:

```
Client A = Virtual Substation A
Client B = Virtual Substation B
Client C = Virtual Substation C
Client D = Virtual Substation D
```

Server

Coordinates training and aggregation.

Global model

The shared model stored by the federated server.

Local model

A copy of the global model trained using one client's local data.

Round

One cycle of:

```
model distribution
→ local training
→ update submission
→ aggregation
```

Non-IID data

The clients have different data distributions.

Example:

```
A: residential load
B: high-demand load
C: solar and EV load
D: stable industrial-like load
```

This is more realistic than giving every client identical data.

11.4 Federated Averaging

A common aggregation method is Federated Averaging:

$$w_{global} = \sum_{k=1}^K \frac{n_k}{n} w_k$$

Where:

- w_k is the model from client k .
- n_k is the number of training samples at client k .
- n is the total number of samples.
- w_{global} is the new global model.

In simple words, a client with more training data receives more influence in the global model.

11.5 Basic federated-training flow

```
Round 1:
Server creates model v1
Server sends v1 to A, B, C, D
```

```
Each client:
- loads private local data
- trains locally
- evaluates locally
- sends model update
```

```
Server:
- validates updates
- aggregates updates
- creates model v2
```

Repeat for several rounds.

11.6 Learn Flower or another framework

For a beginner-friendly implementation, Flower is a practical choice because it provides client-server abstractions and examples for several machine-learning frameworks.^[1] ^[2]

Learn:

- Client class.
- Server.
- Strategy.
- `fit`.
- `evaluate`.
- Model parameter conversion.
- Client sampling.
- Round configuration.
- Metrics aggregation.
- Custom aggregation.

11.7 Important experiment design

You should compare at least:

Experiment	Description
Centralized model	All client data combined
Local-only models	Each substation trains independently
Federated model	Clients train locally and server aggregates
Optional secure model	Aggregation includes update filtering

This allows you to answer:

Does Federated Learning produce a useful global model when substations have different data?

11.8 Federated Learning limitation

Do not claim that Federated Learning automatically guarantees complete privacy.

Model updates can sometimes leak information. For a basic FYP, write:

"The proposed system reduces the need to share raw data; stronger privacy mechanisms such as secure aggregation or differential privacy may be added as future improvements."

12. Phase 10: Explainable AI

An advisor may ask:

"Why should we trust the model's failure prediction?"

A prediction without an explanation is difficult for an operator to act upon.

12.1 What SHAP does

SHAP assigns contributions to input features.

Example:

```
Base failure risk: 20%

Temperature contribution: +31%
Vibration contribution: +24%
Load contribution: +19%
Age contribution: +13%
Voltage contribution: +4%

Final risk: 87%
```

The exact values are generated by the explanation model and should not be invented manually in the final system.

SHAP's `Explainer` interface is designed to explain machine-learning models or prediction functions using Shapley-value-based explanations. ^{[6] [7]}

12.2 Learn these explanation outputs

- Feature importance.
- Local explanation.
- Global explanation.
- Positive contribution.
- Negative contribution.
- Waterfall plot.

- Bar chart.
- Dependence plot.

12.3 Practical model compatibility

Tree-based models such as Random Forest and gradient-boosting models are usually easier to explain than complex deep-learning models.

For your first version:

```
Random Forest predictive-maintenance model  
+ SHAP explanation
```

This is easier to demonstrate than an unexplained deep neural network.

13. Phase 11: Cybersecurity and federated security

Your project can include security, but keep the scope manageable.

13.1 Data-level attacks

Simulate:

- Voltage manipulation.
- Current manipulation.
- Sudden power spikes.
- Missing readings.
- Repeated readings.
- Delayed readings.
- Impossible combinations of values.

Example:

```
Normal voltage: 220-240 V  
Injected voltage: 180 V
```

The anomaly detector should flag the abnormal reading.

13.2 Model-level attacks

A malicious client may send a bad model update.

For example:

```
Client A: normal update  
Client B: normal update
```

```
Client C: normal update
Client D: manipulated update
```

A simple defense can compare updates using:

- Update norm.
- Distance from previous global model.
- Distance from other client updates.
- Gradient clipping.
- Median aggregation.
- Trimmed mean aggregation.

13.3 Security Agent

The Security Agent can monitor:

```
client_id
round_number
update_norm
update_distance
anomaly_score
reputation_score
```

Example output:

```
Client D
Security status: Suspicious
Reason: model update differs significantly from other clients
Action: exclude update from current aggregation
```

Do not claim that this is complete cybersecurity protection. Present it as an experimental malicious-update detection mechanism.

14. Phase 12: Agents and agentic architecture

You should first build normal AI predictions. Add agents afterward.

14.1 What is an agent in this project?

An agent is a software component that:

1. Receives information.
2. Performs a specialized task.
3. Produces a result.
4. May trigger or recommend an action.

Your agents do not need to be autonomous robots or human-like chatbots.

14.2 Recommended agents

Forecast Agent

Input:

```
historical load
weather or time features, if available
substation ID
```

Output:

```
predicted future demand
```

Fault Agent

Input:

```
voltage
current
power
frequency
anomaly score
```

Output:

```
normal, warning, or critical
```

Maintenance Agent

Input:

```
temperature
vibration
load
equipment age
```

Output:

```
health score
failure risk
maintenance recommendation
```

Security Agent

Input:

```
sensor anomaly information  
client model-update statistics
```

Output:

```
normal or suspicious
```

Decision Agent

Input:

```
forecast result  
fault result  
maintenance result  
security result
```

Output:

```
priority  
recommended action  
explanation
```

14.3 Avoid overcomplicating agents

You do not need a large language model for every agent.

A reliable first design is rule-based orchestration:

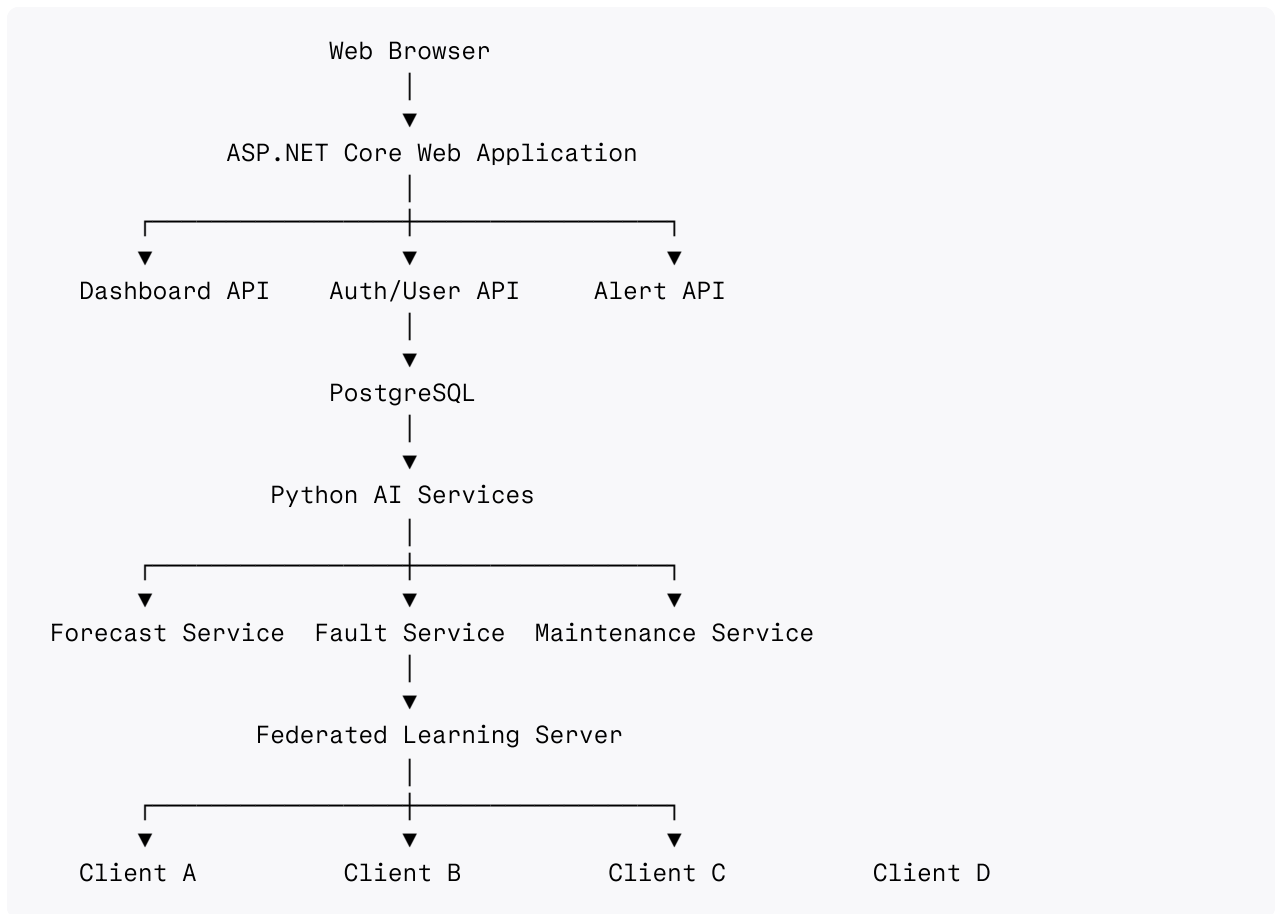
```
if failure_probability >= 0.80:  
    recommendation = "Schedule urgent inspection"  
elif anomaly_score >= 0.90:  
    recommendation = "Investigate abnormal sensor behavior"  
elif predicted_load > capacity_limit:  
    recommendation = "Prepare load-management action"  
else:  
    recommendation = "Continue monitoring"
```

You can later add a language model only for converting structured results into a readable report. The actual safety-related decision should remain based on explicit thresholds and model outputs.

15. Phase 13: Backend and system architecture

Because you already know ASP.NET Core, use it for the main application.

15.1 Recommended high-level architecture



15.2 Responsibilities of ASP.NET Core

Use ASP.NET Core for:

- User authentication.
- Dashboard pages.
- Substation management.
- Alert management.
- Equipment records.
- Report generation.
- API endpoints.
- Database access.
- Role-based access.

Possible roles:

```
Admin
Operator
Advisor/Demo Viewer
```

15.3 Responsibilities of Python

Use Python for:

- Dataset processing.
- Model training.
- Model evaluation.
- Inference.
- Federated client logic.
- SHAP explanations.
- Synthetic-data generation.

15.4 Communication between services

Use REST APIs initially.

Example:

```
POST /api/predictions/load
```

Request:

```
{
  "substationId": "C",
  "features": {
    "previousLoad": 82.5,
    "voltage": 230.1,
    "current": 5.4,
    "hour": 14
  }
}
```

Response:

```
{
  "predictedLoad": 90.8,
  "unit": "kW",
  "status": "increasing"
}
```

For a prototype, REST is easier than introducing Kafka or complex message brokers.

16. Phase 14: Database design

You already have experience with ERP database design, so treat this as a normal relational system.

16.1 Suggested tables

Substations

```
Id
Code
Name
Type
Location
Status
CreatedAt
```

Equipment

```
Id
SubstationId
EquipmentCode
EquipmentType
InstallationDate
Capacity
Status
```

SensorReadings

```
Id
SubstationId
EquipmentId
Timestamp
ActivePower
ReactivePower
Voltage
Current
Frequency
Temperature
Vibration
```

ForecastResults

```
Id
SubstationId
Timestamp
ForecastHorizon
ActualValue
```

PredictedValue
ModelVersion

AnomalyAlerts

Id
SubstationId
EquipmentId
Timestamp
AnomalyScore
Severity
Reason
Status

MaintenancePredictions

Id
EquipmentId
Timestamp
FailureProbability
HealthState
RecommendedAction
ModelVersion

FederatedRounds

Id
RoundNumber
ModelName
ClientCount
GlobalMetric
StartedAt
CompletedAt

ClientUpdates

Id
FederatedRoundId
ClientId
SamplesUsed
UpdateNorm
SecurityScore
Accepted

AgentRecommendations

```
Id
SubstationId
EquipmentId
AgentType
Priority
Recommendation
CreatedAt
ResolvedAt
```

16.2 Important design principle

Do not store every high-frequency raw reading permanently in your transactional database if the volume becomes too large.

Use:

```
Raw CSV/Parquet files for research data
PostgreSQL for metadata, results, alerts, and dashboard data
```

For a small demonstration, storing reduced readings in PostgreSQL is acceptable.

17. Phase 15: Dashboard design

The dashboard is what your advisors will see.

17.1 Main dashboard

Show:

```
Total substations: 4
Normal substations: 2
Warning substations: 1
Critical substations: 1
Active alerts: 3
Last federated round: 12
Global model score: 0.91
```

17.2 Substation map or cards

Each card should show:

```
Substation A
Type: Residential
Current load: 82 kW
Predicted load: 91 kW
Health: Normal
```

Use colors carefully:

```
Green   = Normal
Yellow  = Warning
Red     = Critical
Blue    = Informational
```

17.3 Forecast page

Display:

- Actual load line.
- Predicted load line.
- Forecast horizon.
- MAE and RMSE.
- Substation filter.
- Model version.
- Last update time.

17.4 Anomaly page

Display:

- Anomaly timeline.
- Anomaly score.
- Voltage/current/power chart.
- Severity.
- Explanation.
- Acknowledge button.

17.5 Equipment-health page

Display:

- Transformer ID.
- Temperature trend.
- Vibration trend.
- Load trend.
- Health state.
- Failure probability.
- Maintenance recommendation.
- SHAP feature-importance chart.

17.6 Federated-learning page

Display:

- Current training round.
- Participating clients.
- Local metrics.
- Global metrics.
- Accepted/rejected updates.
- Training loss over rounds.
- Comparison with centralized learning.

17.7 Security page

Display:

- Client update status.
- Suspicious client.
- Update distance.
- Security score.
- Attack simulation status.
- Action taken.

18. The complete operational data flow

18.1 Historical-training flow

```
Download public dataset
↓
Store raw files
↓
Clean and normalize data
↓
Create features
↓
Partition into virtual substations
↓
Train baseline centralized models
↓
Train local models
↓
Run federated rounds
↓
Save model metrics
↓
Save final models
```

18.2 Live-simulation flow

```
Simulation clock starts
↓
Generate/read next sensor record
↓
Send record to local substation client
↓
Local AI service calculates predictions
↓
Anomaly and maintenance checks run
↓
Agents interpret results
↓
Decision Agent creates recommendation
↓
ASP.NET Core stores result
↓
Dashboard refreshes
↓
Operator sees alert
```

18.3 Fault-demo flow

```
Operator clicks "Inject overheating fault"
↓
Temperature begins increasing
↓
Vibration begins increasing
↓
Maintenance model calculates higher risk
↓
Fault Agent detects abnormal behavior
↓
Decision Agent raises priority
↓
Dashboard shows warning/critical state
↓
SHAP displays contributing features
```

19. What you should build first

Do not start by building the full architecture. Start with a small vertical slice.

Milestone 1: Load prediction

Build one Python notebook or script that:

```
reads UCI data
cleans it
creates features
```

```
trains a model
predicts future load
plots actual versus predicted values
```

Milestone 2: Four clients

Create four folders:

```
data/client_a
data/client_b
data/client_c
data/client_d
```

Train an independent model for each.

Milestone 3: Federated prototype

Use a very small dataset and a simple model:

```
server
client A
client B
client C
client D
```

Run five federated rounds.

Milestone 4: Anomaly detection

Inject a voltage or power anomaly and display a generated alert.

Milestone 5: Equipment health

Generate synthetic transformer records, train a classifier, and display failure risk.

Milestone 6: Agents

Add rule-based agents that interpret the model outputs.

Milestone 7: API and dashboard

Connect the Python services to your [ASP.NET](#) Core application.

Milestone 8: Evaluation

Compare:

```
centralized model
local-only models
```

federated model

Then measure performance and communication behavior.

20. What you should learn deeply versus lightly

Area	Required depth
Python, pandas, NumPy	Deep
Data cleaning and feature engineering	Deep
Machine-learning fundamentals	Deep
Time-series forecasting	Deep
Federated Learning	Deep
Basic smart-grid concepts	Medium
Transformer electrical engineering	Basic to medium
Predictive maintenance	Medium
Anomaly detection	Medium to deep
SHAP	Medium
Cybersecurity	Basic to medium
Agent architecture	Medium
<u>ASP.NET</u> Core APIs and dashboard	Deep
Database design	Deep
Advanced power-flow simulation	Optional
Large language models	Optional
Kubernetes and cloud scaling	Not required initially

The most important areas for your thesis are:

Data preparation
+ AI evaluation
+ Federated Learning
+ System integration

21. What not to learn at the beginning

Avoid starting with:

- Advanced electrical power-flow mathematics.
- Reinforcement learning.

- Large language model fine-tuning.
- Kubernetes.
- Blockchain.
- Real-time industrial protocols.
- Complex cryptography.
- Large-scale cloud deployment.
- Transformer fault physics.
- Multiple deep-learning architectures.

These topics can consume your entire FYP without improving your first working prototype.

22. Recommended technology stack

Because of your existing background, this stack is practical:

Layer	Recommended technology
Data processing	Python, pandas, NumPy
Traditional ML	scikit-learn
Deep learning	PyTorch
Federated Learning	Flower
Explainability	SHAP
AI API	FastAPI
Main backend	ASP.NET Core Web API/MVC
Database	PostgreSQL
Dashboard	ASP.NET Razor/MVC, React, or Chart.js
Development	Visual Studio Code, Visual Studio, Jupyter
Deployment	Docker Compose
Version control	Git and GitHub

You can use [ASP.NET](#) Core as the complete web layer and Python as a separate AI service. That lets you use your existing C# skills while still using the Python ecosystem for machine learning.

23. How to explain the project to advisors

Use this simple explanation:

"Our system simulates four substations with different local electricity data. Each substation trains AI models locally for demand forecasting, anomaly detection, and transformer-health prediction. A federated server combines the model updates without collecting raw data. Specialized agents then interpret the predictions and recommend actions. A web dashboard displays the grid status, forecasts, alerts, equipment health, federated-training progress, and explanations."

If they ask what is real and what is simulated, answer honestly:

Real/public:

- Historical electricity behavior
- Solar or household consumption, where available

Simulated:

- Four virtual substations
- Transformer sensor readings
- Equipment degradation
- Faults
- Cyberattacks
- Live monitoring process

That separation increases your credibility.

24. Minimum viable version

If your time or team size is limited, build this version:

```
UCI electricity dataset
  ↓
Four virtual clients
  ↓
One demand-forecasting model
  ↓
Federated Averaging
  ↓
One synthetic transformer-health model
  ↓
One anomaly-injection scenario
  ↓
SHAP explanation
  ↓
ASP.NET Core dashboard
```

This is already a strong FYP.

The additional solar, EV, model-poisoning, multiple agents, and advanced security modules should be treated as extensions.

25. Final learning target

At the end, you should be able to answer these questions:

1. What is a smart grid?
2. What is a substation?
3. What data does each client own?
4. Why are the clients non-IID?
5. What is the difference between centralized and federated learning?
6. What happens during one federated round?
7. How is the global model aggregated?
8. How do you forecast future demand?
9. How do you detect an anomaly?
10. How do you generate and label synthetic equipment faults?
11. How do you evaluate the models?
12. How does SHAP explain a prediction?
13. How can a malicious federated update be detected?
14. What does each agent do?
15. How does the dashboard receive and display results?
16. Which components are real and which are simulated?
17. What are the limitations of your system?
18. Why is the project useful if it is not connected to a real electrical grid?

If you can explain and demonstrate those points, you will understand the project well enough to implement and defend it.

The best practical path for you is:

```
Python and data
→ centralized ML
→ forecasting
→ anomaly detection
→ synthetic maintenance data
→ federated learning
→ SHAP
→ agents
→ ASP.NET Core dashboard
→ experiments and evaluation
```

✱

1. <https://flower.ai/docs/framework/tutorial-series-what-is-federated-learning.html>

2. <https://github.com/StefanDurhamUK/FL-Flower>
3. <https://www.nrel.gov/grid/smart-ds>
4. <https://www.nrel.gov/grid/all-data-tools>
5. <https://energy.acm.org/resources/>
6. <https://shap.readthedocs.io/en/latest/generated/shap.Explainer.html>
7. [https://shap.readthedocs.io/en/latest/example_notebooks/overviews/An introduction to explainable AI with Shapley values.html](https://shap.readthedocs.io/en/latest/example_notebooks/overviews/An%20introduction%20to%20explainable%20AI%20with%20Shapley%20values.html)
8. <https://www.nrel.gov/grid/news/program/2020/synthetic-data-sets-set-new-standard>
9. <https://data.nrel.gov/>
10. <https://github.com/adap/flower>
11. <https://github.com/flwrllabs/flower>
12. https://github.com/Maryamr92/FedAirNet_flower
13. <https://github.com/open-energy-transition/Awesome-Grid-Model-Data>
14. <https://catalog.data.gov/?keyword=load>
15. [https://catalog.data.gov/dataset/?tags=grid&publisher=National+Renewable+Energy+Laboratory+\(NREL\)](https://catalog.data.gov/dataset/?tags=grid&publisher=National+Renewable+Energy+Laboratory+(NREL))